Họ và tên: Nguyễn Đức Bảo Trung
MSSV: 25521963
Mã lớp: IT007.R16.1

OPERATING SYSTEM

LAB 3 REPORT

SUMMARY:

Task		Tatus	Page
Section 3.5	Bài 1	Done	2
	BTH 1	Done	4
	BTH 2	Done	5
	BTH 3	Done	14

Self score: 10/10

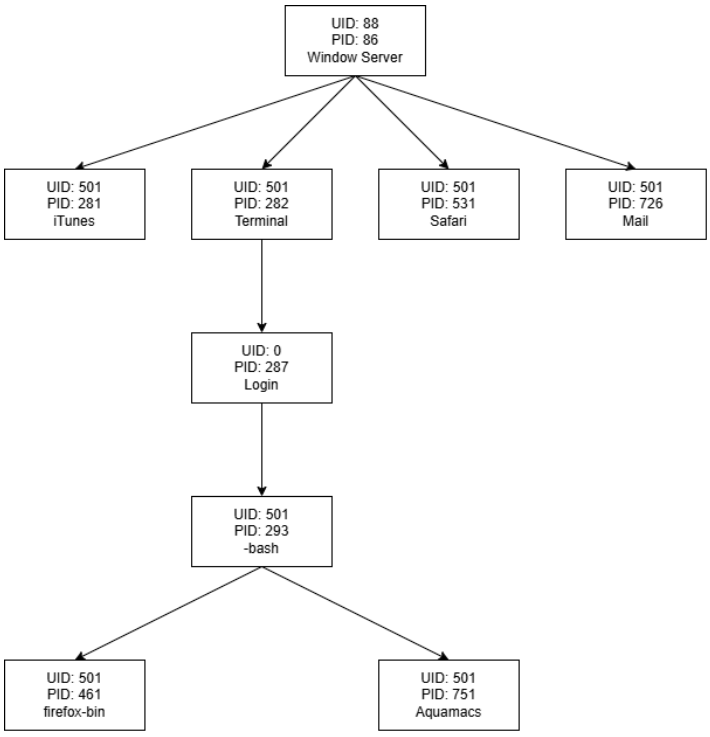
3.5 Bài tập ôn tập

1. Mối quan hệ cha-con giữa các tiến trình

a. Vẽ cây quan hệ parent-child của các tiến trình bên dưới:

UID	PID	PPID	COMMAND
88	86	1	WindowServer
501	281	86	iTunes
501	282	86	Terminal
0	287	282	login
501	461	293	firefox-bin
501	531	86	Safari
501	726	86	Mail
501	751	293	Aquamacs
501	293	287	-bash

Vẽ cây quan hệ parent – child của các tiếng trình:



Hình 1.1 Cây tiến trình

b. Trình bày cách sử dụng lệnh ps để tìm tiến trình cha của một một tiến trình dựa vào PID của nó.

- Giống như top, ps cũng là lệnh giúp hiển thị chi tiết của tiến trình, trong đó PID là ID của tiến trình và PPID là PID của tiến trình cha của tiến trình đó.

- Để sử dụng lệnh ps tìm tiến trình cha của một tiến trình dựa vào PID của nó ta dùng lệnh: **ps -f**

```
NguyenDucBaoTrung@25521963:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
NguyenD+   38762   38661  0 09:04 pts/0    00:00:00 /usr/bin/bash
NguyenD+   38799   38762 50 09:05 pts/0    00:00:00 ps -f
NguyenDucBaoTrung@25521963:~$
```

Hình 1.2: Kết quả khi chạy lệnh ps -f

- Trong hình ta có thể thấy rằng tiến trình bash có PID là 38762 là tiến trình cha của tiến trình ps -f nhờ lệnh ps trong qua PID và PPID của chúng.

- Ta có thể sử dụng lệnh ps -fp [pidlist] để tìm PPID của tiến trình thông qua PID của tiến đó

```
NguyenDucBaoTrung@25521963:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
NguyenD+   38762   38661  0 09:04 pts/0    00:00:00 /usr/bin/bash
NguyenD+   38799   38762 50 09:05 pts/0    00:00:00 ps -f
NguyenDucBaoTrung@25521963:~$ ps -fp 38762
UID          PID    PPID  C STIME TTY          TIME CMD
NguyenD+   38762   38661  0 09:04 pts/0    00:00:00 /usr/bin/bash
NguyenDucBaoTrung@25521963:~$
```

Hình 1.3: Kết quả khi chạy lệnh ps -fp 38762

c. Tìm hiểu và cài đặt lệnh pstree (nếu chưa được cài đặt), sau đó trình bày cách sử dụng lệnh này để tìm tiến trình cha của một tiến trình dựa vào PID của nó.

- Lệnh pstree giúp hiển thị các tiến trình dưới dạng sơ đồ cây. Để sử dụng lệnh này để tìm tiến trình cha của một tiến trình dựa vào PID của nó ta dùng lệnh pstree -sg [pidlist]

```
NguyenDucBaoTrung@25521963:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
NguyenD+   38762   38661  0 09:04 pts/0    00:00:00 /usr/bin/bash
NguyenD+   38799   38762 50 09:05 pts/0    00:00:00 ps -f
NguyenDucBaoTrung@25521963:~$ ps -fp 38762
UID          PID    PPID  C STIME TTY          TIME CMD
NguyenD+   38762   38661  0 09:04 pts/0    00:00:00 /usr/bin/bash
NguyenDucBaoTrung@25521963:~$ pstree -sg 38762
systemd(1)───systemd(37118)───ptyxis(37714)───ptyxis-agent(38661)───bash(38762)───pstree(38825)
NguyenDucBaoTrung@25521963:~$
```

Hình 1.4: Kết quả khi chạy lệnh pstree -sg 38762

- Qua kết quả trả về của lệnh pstree ta có thể thấy được tiến trình cha của bash (PID 38762) là tiến trình ptyxis-agent-(PID 38661).

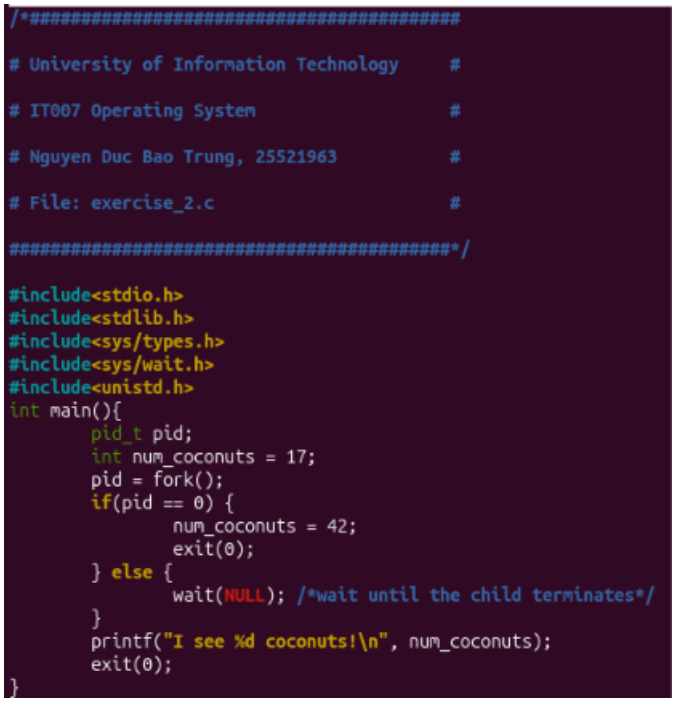
BTH 1: Chương trình bên dưới in ra kết quả gì? Giải thích tại sao? (Bài 2, mục 3.5)

```
/*#######################################
# University of Information Technology      #
# IT007 Operating System                    #
# <Your name>, <your Student ID>            #
# File: exercise_2.c                        #
#######################################*/

#include<stdio.h>

int main(){
   pid_t pid;
   int num_coconuts = 17;
   pid = fork();
   if(pid == 0) {
      num_coconuts = 42;
      exit(0);
   } else {
      wait(NULL); /*wait until the child terminates */
   }
   printf("I see %d coconuts!\n", num_coconuts);
   exit(0);
}
```

- File script của exercise_2.c:



```
/*########################################

# University of Information Technology     #

# IT007 Operating System                   #

# Nguyen Duc Bao Trung, 25521963           #

# File: exercise_2.c                       #

########################################*/

#include<stdio.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/wait.h>
#include<unistd.h>
int main(){
        pid_t pid;
        int num_coconuts = 17;
        pid = fork();
        if(pid == 0) {
                num_coconuts = 42;
                exit(0);
        } else {
                wait(NULL); /*wait until the child terminates*/
        }
        printf("I see %d coconuts!\n", num_coconuts);
        exit(0);
}
```

Hình 1.1: Đoạn mã trong file exercise_2.c

- Chạy lại chương trình:

```
NguyenDucBaoTrung@25521963:~$ nano exercise_2.c
NguyenDucBaoTrung@25521963:~$ gcc exercise_2.c -o exercise_2
NguyenDucBaoTrung@25521963:~$ ./exercise_2
I see 17 coconuts!
NguyenDucBaoTrung@25521963:~$
```

Hình 1.2: Kết quả chạy chương trình

- Vì tiến trình con có dòng lệnh "exit(0);" khi tiến trình con chạy giá trị của "num_coconut" đã thay đổi thành 42, nhưng vì dòng lệnh trên nên dòng lệnh in kết quả ra màn hình không thực hiện được, nếu xóa dòng "exit(0);" này đi ta sẽ có kết quả như sau:

```
NguyenDucBaoTrung@25521963:~$ nano exercise_2.c
NguyenDucBaoTrung@25521963:~$ gcc exercise_2.c -o exercise_2
NguyenDucBaoTrung@25521963:~$ ./exercise_2
I see 42 coconuts!
I see 17 coconuts!
NguyenDucBaoTrung@25521963:~$
```

Hình 2.2: Kết quả chạy chương trình sau khi xóa dòng lệnh "exit(0);" trong tiến trình con.

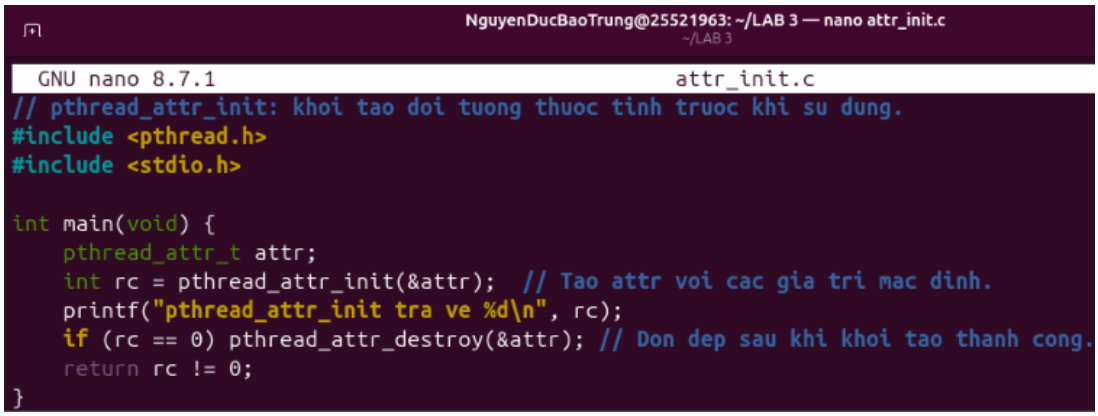
BTH 2: Trong phần thực hành, các ví dụ chỉ sử dụng thuộc tính mặc định của pthread, hãy tìm hiểu POSIX thread và trình bày tất cả các hàm được sử dụng để làm thay đổi thuộc tính của pthread, sau đó viết các chương trình minh họa tác động của các thuộc tính này và chú thích đầy đủ cách sử dụng hàm này trong chương trình. (Gợi ý các hàm liên quan đến thuộc tính của pthread đều bắt đầu bởi: pthread_attr_*) (Bài 3, mục 3.5)

Hàm	Chức năng
Pthread_attr_init	Khởi tạo giá trị mặc định cho đối tượng thuộc tính
Pthread_attr_destroy	Xóa bộ nhớ được cấp phát trong quá trình khởi tạo
Pthread_attr_getschedparam	Trả về các tham số lịch trình (Scheduling Parameter) được xác định bởi pthread_attr_setschedparam ()
Pthread_attr_getschedpolicy	Để xuất scheduling policy của thread
Pthread_attr_getdetachstate	Lấy truy xuất trạng thái khởi tạo của thread, có thể thể là riêng lẻ hoặc kết hợp
Pthread_attr_getinheritsched	Trả về chính sách lịch trình (scheduling policy) được set bởi pthread_attr_setinheritsched ().
Pthread_attr_getscope	Truy xuất phạm vi của thread
Pthread_attr_setdetachstate	Sử dụng lại ID và tài nguyên của thread khi nó bị ngắt mà không phải chờ nếu thread có thuộc tính riêng lẻ.
Pthread_attr_setguardsize	Set kích thước của khu vực an toàn của thread
Pthread_attr_setstackaddr	Set địa chỉ stack của thread
Pthread_attr_setstacksize	Set kích thước stack của thread

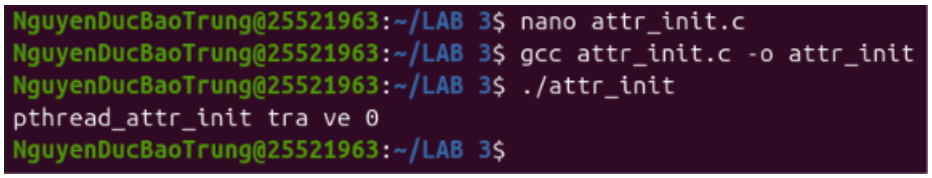
Pthread_attr_getguardsize	Lấy kích thước của khu vực an toàn của thread
Pthread_attr_getstackaddr	Trả về địa chỉ stack của thread được set bởi pthread_attr_setstackaddr ()
Pthread_attr_getstacksize	Trả về kích thước stack của thread được set bởi pthread_attr_setstacksize ().

Pthread_attr_init

- File script của attr_init.c



- Kết quả chạy file:



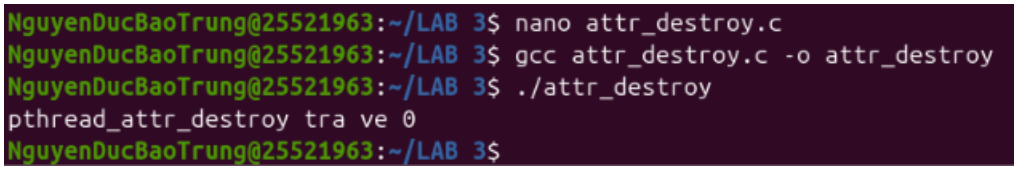
- Giải thích kết quả chạy: Hàm pthread_attr_init() trả về 0, chứng tỏ đối tượng thuộc tính attr đã được khởi tạo thành công với các giá trị mặc định.

Pthread_attr_destroy

- File script của attr_destroy.c



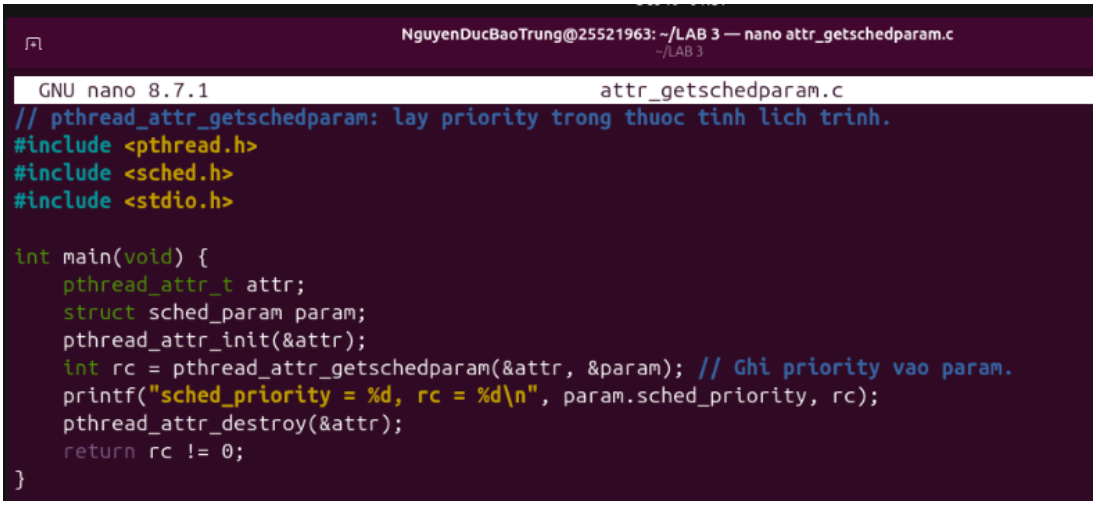
- Kết quả chạy file:



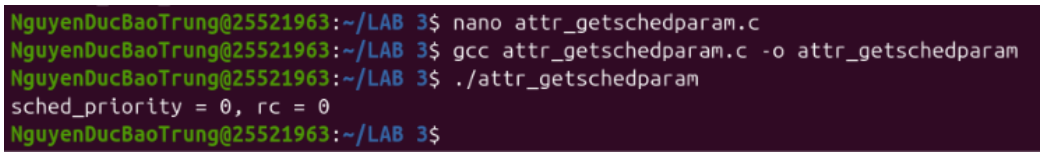
- Giải thích kết quả chạy: Hàm pthread_attr_destroy() trả về 0, chứng tỏ đối tượng thuộc tính đã được hủy thành công sau khi sử dụng.

Pthread_attr_getschedparam

- File script của attr_getschedparam



- Kết quả chạy file:



- Giải thích kết quả chạy: Kết quả cho biết độ ưu tiên mặc định (sched_priority) của thread trong thuộc tính attr. Với chính sách SCHED_OTHER, giá trị này thường là 0.

Pthread_attr_getschedpolicy

- File script của attr_getschedpolicy



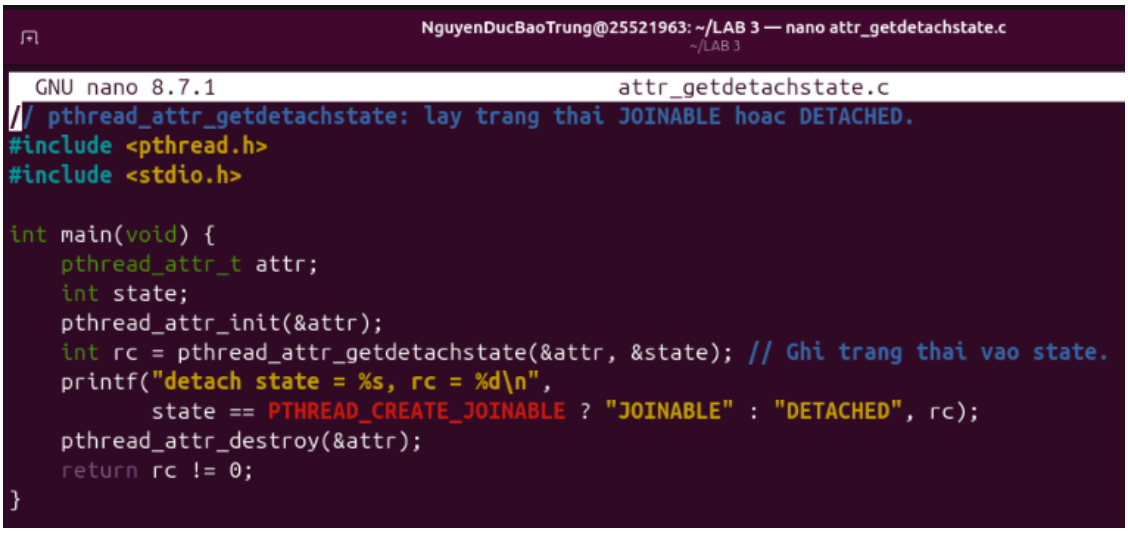
- Kết quả chạy file:



- Giải thích kết quả chạy: Kết quả cho biết chính sách lập lịch mặc định của thread. Giá trị 0 tương ứng với SCHED_OTHER, là chính sách lập lịch thông thường của Linux.

Pthread_attr_getdetachstate

- File script của attr_getdetachstate



- Kết quả chạy file:



- Giải thích kết quả chạy: Kết quả cho biết thread mới được tạo mặc định ở trạng thái JOINABLE, nghĩa là thread cha có thể gọi pthread_join() để chờ thread con kết thúc.

Pthread_attr_getinheritsched

- File script của attr_getinheritsched



- Kết quả chạy file:

```
NguyenDucBaoTrung@25521963:~/LAB 3$ nano attr_getinheritsched.c
NguyenDucBaoTrung@25521963:~/LAB 3$ gcc attr_getinheritsched.c -o attr_getinheritsched
NguyenDucBaoTrung@25521963:~/LAB 3$ ./attr_getinheritsched
inherit sched = INHERIT_SCHED, rc = 0
NguyenDucBaoTrung@25521963:~/LAB 3$
```

- Giải thích kết quả chạy: Kết quả cho biết thread mới mặc định kế thừa chính sách và tham số lập lịch từ thread tạo ra nó (PTHREAD_INHERIT_SCHED).

Pthread_attr_getscope

- File script của attr_getscope



- Kết quả chạy file:

```
NguyenDucBaoTrung@25521963:~/LAB 3$ nano attr_getscope.c
NguyenDucBaoTrung@25521963:~/LAB 3$ gcc attr_getscope.c -o attr_getscope
NguyenDucBaoTrung@25521963:~/LAB 3$ ./attr_getscope
scope = PTHREAD_SCOPE_SYSTEM, rc = 0
NguyenDucBaoTrung@25521963:~/LAB 3$
```

- Giải thích kết quả chạy: Kết quả cho biết thread dùng PTHREAD_SCOPE_SYSTEM, tức thread cạnh tranh CPU với các thread khác trên toàn hệ thống.

Pthread_attr_setdetachstate

- File script của attr_setdetachstate

NguyenDucBaoTrung@25521963: ~/LAB 3 — nano attr_setdetachstate.c
~/LAB 3

GNU nano 8.7.1 attr_setdetachstate.c

```c
// Minh hoa pthread_attr_setdetachstate: dat thread o che do DETACHED.
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static void *worker(void *arg) {
    (void)arg;
    puts("Thread da chay xong.");
    return NULL;
}

int main(void) {
    pthread_attr_t attr;
    pthread_t tid;
    int state, rc;

    pthread_attr_init(&attr);                          // Tao doi tuong thuoc tinh.
    pthread_attr_getdetachstate(&attr, &state);
    printf("Truoc: %s\n", state == PTHREAD_CREATE_JOINABLE ? "JOINABLE" : "DETACHED");

    rc = pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
    if (rc != 0) return printf("Loi setdetachstate: %d\n", rc), 1;
    pthread_attr_getdetachstate(&attr, &state);        // Kiem tra gia tri vua dat.
    printf("Sau: %s\n", state == PTHREAD_CREATE_JOINABLE ? "JOINABLE" : "DETACHED");

    pthread_create(&tid, &attr, worker, NULL);         // Thread tao ra se tu giai phong tai nguyen.
    pthread_attr_destroy(&attr);
    sleep(1);                                          // Khong pthread_join() voi detached thread.
    return 0;
}
```

- Kết quả chạy file:

```
NguyenDucBaoTrung@25521963:~/LAB 3$ nano attr_setdetachstate.c
NguyenDucBaoTrung@25521963:~/LAB 3$ gcc attr_setdetachstate.c -o attr_setdetachstate
NguyenDucBaoTrung@25521963:~/LAB 3$ ./attr_setdetachstate
Truoc: JOINABLE
Sau: DETACHED
Thread da chay xong.
NguyenDucBaoTrung@25521963:~/LAB 3$
```

- Giải thích kết quả chạy: Kết quả cho thấy thuộc tính đã đổi từ JOINABLE sang DETACHED. Thread detached sẽ tự giải phóng tài nguyên khi kết thúc và không cần gọi pthread_join().

Pthread_attr_setguardsize

- File script của attr_setguardsize

NguyenDucBaoTrung@25521963: ~/LAB 3 — nano attr_setguardsize.c
~/LAB 3

GNU nano 8.7.1 attr_setguardsize.c

```c
// Minh hoa pthread_attr_setguardsize: dat vung bao ve o cuoi stack.
#include <pthread.h>
#include <stdio.h>

int main(void) {
    pthread_attr_t attr;
    size_t before, after;
    int rc;

    pthread_attr_init(&attr);
    pthread_attr_getguardsize(&attr, &before);
    rc = pthread_attr_setguardsize(&attr, 4096);       // Dat guard size la 1 trang 4 KiB.
    pthread_attr_getguardsize(&attr, &after);          // Doc lai de chung minh thay doi.
    printf("Guard size: %zu -> %zu bytes (rc = %d)\n", before, after, rc);
    pthread_attr_destroy(&attr);
    return rc != 0;
}
```

- Kết quả chạy file:

```
NguyenDucBaoTrung@25521963:~/LAB 3$ nano attr_setguardsize.c
NguyenDucBaoTrung@25521963:~/LAB 3$ gcc attr_setguardsize.c -o attr_setguardsize
NguyenDucBaoTrung@25521963:~/LAB 3$ ./attr_setguardsize
Guard size: 4096 -> 4096 bytes (rc = 0)
NguyenDucBaoTrung@25521963:~/LAB 3$
```

- Giải thích kết quả chạy: Kết quả cho thấy vùng bảo vệ stack đã được đặt thành 4096 bytes. Vùng này giúp phát hiện lỗi tràn stack. Nếu giá trị trước và sau đều là 4096, đó là vì giá trị mặc định của hệ thống đã trùng với giá trị được đặt.

Pthread_attr_setstackaddr

- File script của attr_setstackaddr

NguyenDucBaoTrung@25521963: ~/LAB 3 — nano attr_setstackaddr.c

GNU nano 8.7.1 attr_setstackaddr.c

```c
// pthread_attr_setstackaddr: dat dia chi stack. API nay da loi thoi trong POSIX.
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    pthread_attr_t attr;
    void *address = malloc(1024 * 1024);
    if (address == NULL) return puts("Khong du bo nho."), 1;
    pthread_attr_init(&attr);
    int rc = pthread_attr_setstackaddr(&attr, address); // Chi dat dia chi; code moi dung pthread_attr_setstack.
    printf("Da dat stack address = %p, rc = %d\n", address, rc);
    pthread_attr_destroy(&attr);
    free(address);
    return rc != 0;
}
```

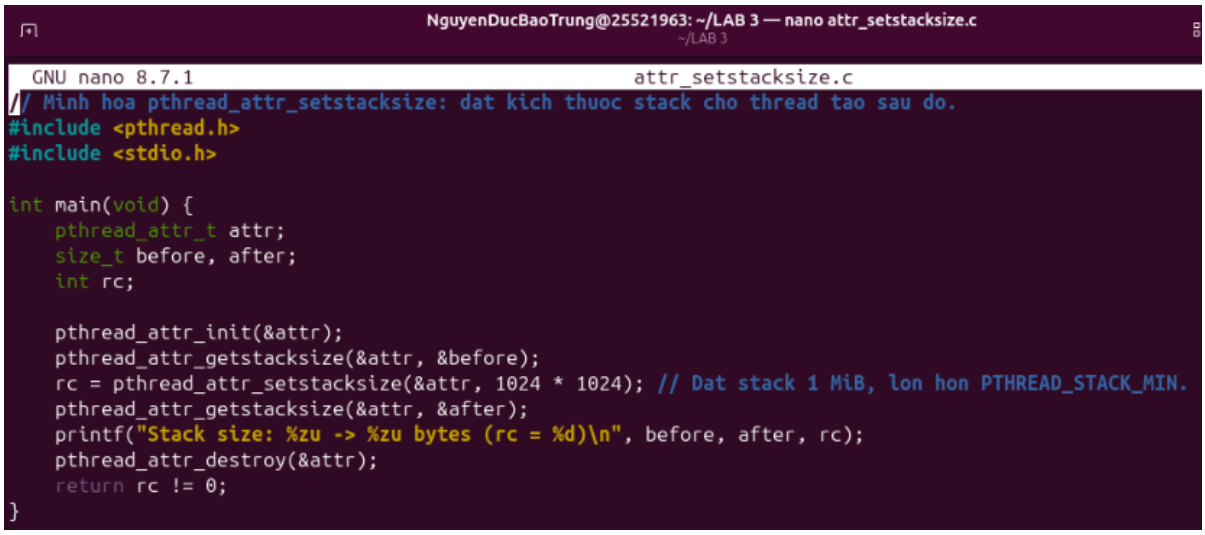
- Kết quả chạy file:

```
NguyenDucBaoTrung@25521963:~/LAB 3$ nano attr_setstackaddr.c
NguyenDucBaoTrung@25521963:~/LAB 3$ gcc attr_setstackaddr.c -o attr_setstackaddr
attr_setstackaddr.c: In function ‘main’:
attr_setstackaddr.c:12:5: warning: ‘pthread_attr_setstackaddr’ is deprecated [-Wdeprecated-declarations]
   12 |     int rc = pthread_attr_setstackaddr(&attr, address); // Chi dat dia chi; code moi dung pthread_attr_setstack.
      |     ^~~
In file included from attr_setstackaddr.c:3:
/usr/include/pthread.h:361:12: note: declared here
  361 | extern int pthread_attr_setstackaddr (pthread_attr_t *__attr,
      |            ^~~~~~~~~~~~~~~~~~~~~~~~~
/usr/bin/x86_64-linux-gnu-ld.bfd: /tmp/ccpt7cRa.o: in function `main':
attr_setstackaddr.c:(.text+0x61): warning: the use of `pthread_attr_setstackaddr' is deprecated, use `pthread_attr_setst
ack'
NguyenDucBaoTrung@25521963:~/LAB 3$ ./attr_setstackaddr
Da dat stack address = 0x70d010cff010, rc = 0
NguyenDucBaoTrung@25521963:~/LAB 3$
```

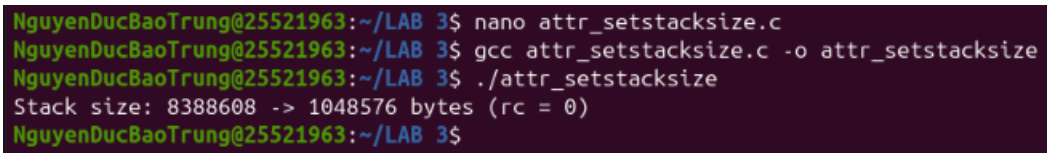
- Giải thích kết quả chạy: Hàm đã đặt địa chỉ stack theo vùng nhớ được cấp phát. Tuy nhiên, khi biên dịch xuất hiện cảnh báo, không phải lỗi, vì pthread_attr_setstackaddr() đã lỗi thời. Trong chương trình POSIX hiện đại nên dùng pthread_attr_setstack() để đặt đồng thời địa chỉ và kích thước stack.

Pthread_attr_setstacksize

- File script của attr_setstacksize



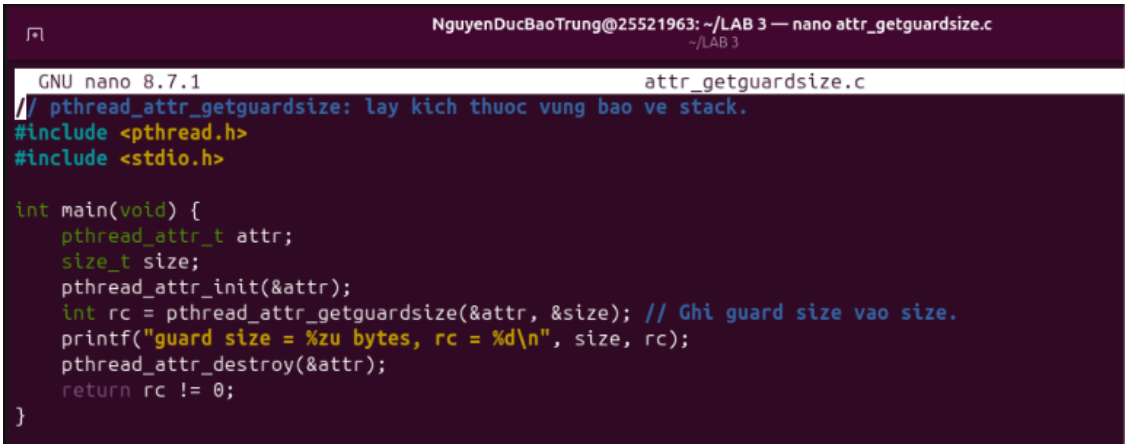
- Kết quả chạy file:



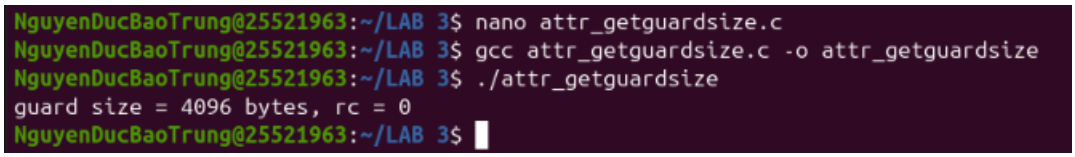
- Giải thích kết quả chạy: Kết quả cho thấy kích thước stack của thread đã được đổi thành 1048576 bytes (1 MiB). Giá trị này sẽ được áp dụng cho thread được tạo với thuộc tính attr.

Pthread_attr_getguardsize

- File script của attr_getguardsize



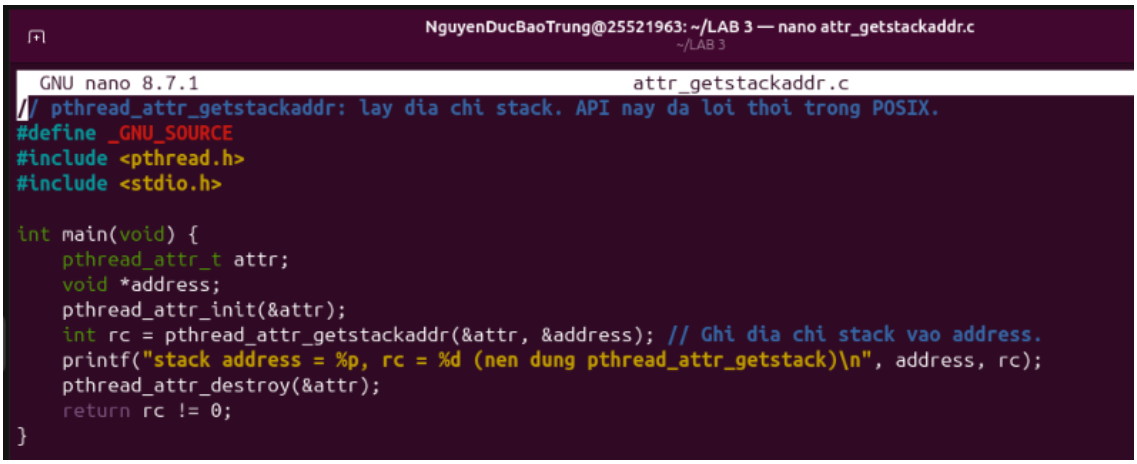
- Kết quả chạy file:



- Giải thích kết quả chạy: Kết quả cho biết kích thước vùng bảo vệ stack mặc định của thread. Vùng bảo vệ này giúp hệ điều hành phát hiện truy cập vượt quá giới hạn stack.

Pthread_attr_getstackaddr

- File script của attr_getstackaddr



- Kết quả chạy file:

```
NguyenDucBaoTrung@25521963:~/LAB 3$ nano attr_getstackaddr.c
NguyenDucBaoTrung@25521963:~/LAB 3$ gcc attr_getstackaddr.c -o attr_getstackaddr
attr_getstackaddr.c: In function ‘main’:
attr_getstackaddr.c:10:5: warning: ‘pthread_attr_getstackaddr’ is deprecated [-Wdeprecated-declarations]
   10 |     int rc = pthread_attr_getstackaddr(&attr, &address); // Ghi dia chi stack vao address.
      |     ^~~
In file included from attr_getstackaddr.c:3:
/usr/include/pthread.h:353:12: note: declared here
  353 | extern int pthread_attr_getstackaddr (const pthread_attr_t *__restrict
      |            ^~~~~~~~~~~~~~~~~~~~~~~~~
/usr/bin/x86_64-linux-gnu-ld.bfd: /tmp/cc0Klry5.o: in function `main':
attr_getstackaddr.c:(.text+0x36): warning: the use of `pthread_attr_getstackaddr' is deprecated, use `pthread_attr_getst
ack'
NguyenDucBaoTrung@25521963:~/LAB 3$ ./attr_getstackaddr
stack address = (nil), rc = 0 (nen dung pthread_attr_getstack)
NguyenDucBaoTrung@25521963:~/LAB 3$
```

- Giải thích kết quả chạy:

Khi biên dịch, chương trình chỉ báo warning chứ không phải lỗi vì pthread_attr_getstackaddr() đã bị POSIX đánh dấu là hàm lỗi thời (deprecated). Hàm này chỉ lấy địa chỉ stack, trong khi cách mới và đầy đủ hơn là dùng pthread_attr_getstack() để lấy đồng thời địa chỉ và kích thước stack.

Chương trình vẫn biên dịch và chạy được. Kết quả stack address = (nil) nghĩa là thuộc tính attr vừa khởi tạo chưa được gán vùng stack riêng; khi tạo thread bằng pthread_create(), hệ điều hành mới tự cấp phát stack mặc định cho thread.

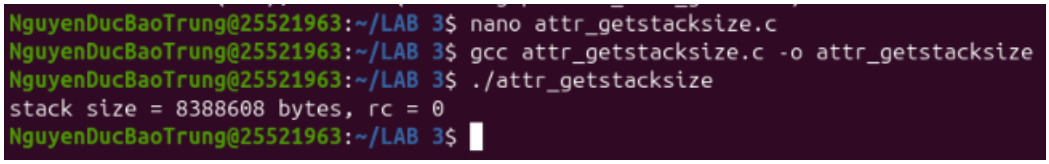
Vì vậy, trong lập trình POSIX hiện đại nên dùng pthread_attr_getstack() thay cho pthread_attr_getstackaddr().

Pthread_attr_getstacksize

- File script của attr_getstacksize



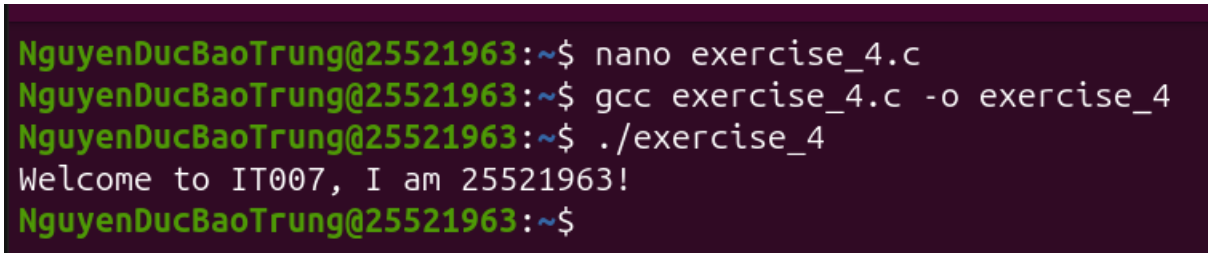
- Kết quả chạy file:



- Giải thích kết quả chạy: Kết quả cho biết kích thước stack mặc định của thread được tạo từ thuộc tính attr. Giá trị này phụ thuộc vào cấu hình hệ thống Linux.

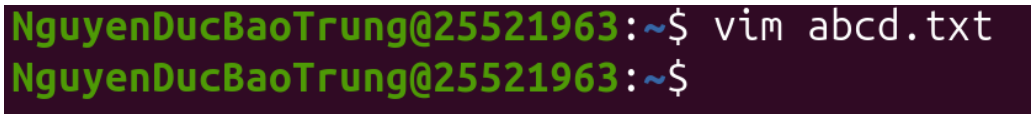
BTH 3: Viết chương trình làm các công việc sau theo thứ tự: (Bài 4, mục 3.5)

a. In ra dòng chữ: “Welcome to IT007, I am <your_Student_ID>!”



Hình 3.1: Kết quả câu a

b. Mở tệp abcd.txt bằng vim editor



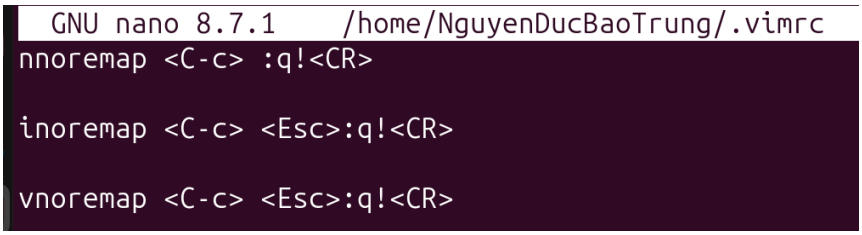
Hình 3.2: Mở file abcd.txt bằng vim



Hình 3.3: Nội dung file abcd.txt khi mở bằng vim

c. Tắt vim editor khi người dùng nhấn CRTL+C

- dùng Key Mapping cấu hình ngay trong file của Vim để làm điều này, khi mở 1 file vim thì ấn Ctrl + C nó sẽ thoát thay vì Esc + : w q



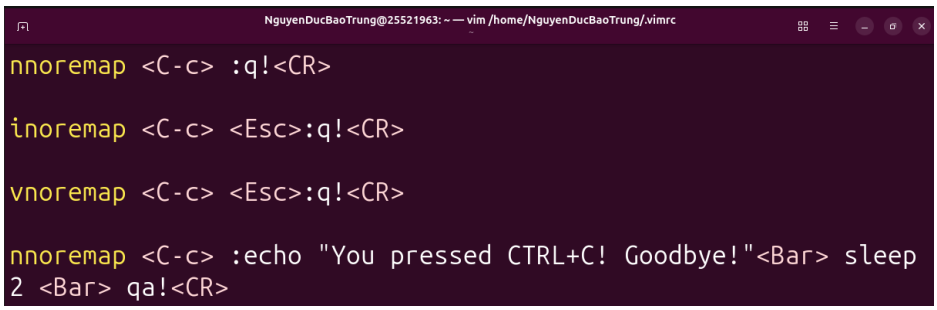
Hình 3.4: Nội dung file .vimrc

- Lệnh đầu tiên: nnoremap <C-c> :q!<CR>
Giải thích: Khi ở Normal mode, nhấn Ctrl+C sẽ thoát thẳng (bỏ qua lưu)

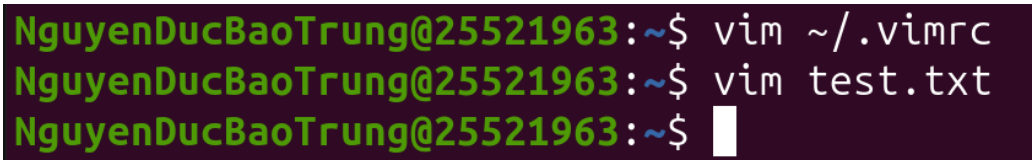
- Lệnh thứ hai: inoremap <C-c> <Esc>:q!<CR>
Giải thích: Khi ở Insert mode (đang gõ văn bản), nhấn Ctrl+C cũng thoát thẳng

- Lệnh thứ ba: vnoremap <C-c> <Esc>:q!<CR>
Giải thích: Khi ở Visual mode (đang bôi đen), nhấn Ctrl+C cũng thoát thẳng

d. Khi người dùng nhấn CTRL+C thì in ra dòng chữ: “You are pressed CTRL+C! Goodbye!”



Hình 3.5: Nội dung file .vimrc



Hình 3.6: Mở file test.txt



Hình 3.7: Sau khi ấn Ctrl + C thì hiển thị dòng đã bấm Ctrl + C